ENGINEERING & PRODUCT ROADMAP

Development Roadmap

Focus & Reminder App | Phased Validation, Architecture, Vibe-Coding & Release Plan

PURPOSE	TARGET APP	APPROACH
Execution Roadmap & Technical Sequencing	Focus & Reminder (Android)	Bounded Increments & Physical Device Validation
CORE TECH	**CURRENT PHASE**	**ARCHITECTURE**
Native Android | Kotlin | Jetpack Compose	Phase 0 (Requirements Consolidation)	Local-First (No Cloud Dependency)

---

1. Guiding approach

Build the product in small, verifiable increments. Use AI to accelerate implementation, but maintain explicit control over requirements, architecture, permissions, testing, and code changes.

The governing rule is:

> **GOVERNING RULE**
>
> **Validate platform feasibility → define architecture → implement one vertical slice at a time → test on a real device → expand.**

Do not attempt to generate the complete application in one prompt.

---

2. Phase roadmap

Phase 0 — Requirements consolidation

Deliverables

- PRD v1.0.
- Open-decision register.
- Initial scope and priority definitions.
- Acceptance criteria.

Actions

1. Review the PRD and resolve the questions that affect the first release.
2. Freeze the MVP scope.
3. Identify which requirements are product goals and which depend on Android permissions.
4. Record known limitations, especially restart bypass and full-screen reminder restrictions.

Exit criteria: The initial requirements are sufficiently clear to test and implement without repeatedly redefining the product.

Phase 1 — Technical feasibility spike

Deliverables

- Android prototype.
- Feasibility report.
- Supported-device assumptions.
- Architecture risk register.

Prototype A: Usage detection and blocking

1. Create a minimal native Android project.
2. Request and verify the required usage access.
3. Detect transitions into and out of a selected app.
4. Track foreground usage and five-minute session continuity.
5. Prototype a blocking presentation.
6. Test what happens when the user navigates home, returns through an app link, changes permissions, force-stops the app, or restarts the phone.
7. Record behaviour across relevant Android versions.

Prototype B: Full-screen reminder

1. Schedule a local reminder.
2. Trigger a notification while another app is in the foreground.
3. Test full-screen presentation eligibility and fallback behaviour.
4. Test lock-screen, silent, and vibrate scenarios.
5. Test reboot recovery and exact-alarm permission requirements.
6. Record what is possible without special privileges.

Exit criteria

- A demonstrated, documented usage-detection strategy.
- A demonstrated blocking strategy with known limitations.
- A demonstrated reminder strategy and fallback.
- No major architectural assumption remains untested.

Do not polish the UI during this phase.

Phase 2 — High-Level Design

Deliverable: HLD document.

Define:

- Major components and their responsibilities.
- Data flow between UI, domain logic, Android services, and local storage.
- State ownership and persistence boundaries.
- Permission dependencies.
- Error-handling strategy.
- Offline behaviour.
- Integration boundaries for future calendar and widget features.

Exit criteria: Each core feature can be mapped to specific components, and dependencies are understood.

Phase 3 — Detailed technical design

Deliverables

- Technical requirements document.

- Detailed component specifications.
- Database schema.
- State-machine definitions.
- API and platform integration notes.
- Test plan.

Define the rolling-window algorithm, cooldown rules, emergency override, reminder scheduling, reboot handling, and permission-loss behaviour before implementing them.

Exit criteria: The next implementation tasks can be given to a coding agent with clear inputs, outputs, and tests.

Phase 4 — Core implementation

Build in this order:

1. Project shell, navigation, and settings.
2. Local data model and persistence.
3. Pure Kotlin rolling-window engine.
4. Usage tracking adapter.
5. Warning and blocking interface.
6. Cooldown and recovery logic.
7. Reflective emergency-access flow.
8. Local reminder creation and scheduling.
9. Reminder presentation and acknowledgement.
10. Integration testing and battery profiling.

Exit criteria: Both core feature tracks work end to end on a physical device.

Phase 5 — Stabilisation and personal-use release

Actions

- Test airplane mode.
- Test app process termination.
- Test reboot behaviour.
- Test permission revocation.
- Test reminder scheduling under battery-saving conditions.
- Test silent and vibrate modes.
- Measure battery consumption.
- Fix critical defects.
- Document limitations and setup instructions.

Exit criteria: The MVP passes the release acceptance criteria in the PRD.

Phase 6 — Calendar integration

Deliverables

- Calendar integration design.
- Permission and data-ownership rules.
- Event-to-reminder mapping.
- Synchronization and conflict strategy.

Implement read-only event display first. Add event write-back and synchronization only after the read path is reliable.

Phase 7 — Daily-planner widget

Deliverables

- Widget UX and layout.
- Data-query and refresh strategy.
- Quick-action design.
- Widget testing across launchers and device sizes.

The widget should reuse the existing reminder and calendar data models.

Phase 8 — Product refinement

Use actual usage and feedback to decide whether to add analytics, configurable policies, history, backup, cross-device support, or other advanced features.

Do not automatically build every item in the long-term vision.

---

3. Required documents and their sequence

Order	Document	Purpose
1	PRD	Defines product requirements and acceptance criteria
2	Feasibility report	Records what Android can support and what must be approximated
3	HLD	Defines major components and their interactions
4	Technical design / LLD	Defines algorithms, data models, states, APIs, and implementation details
5	Test plan	Defines expected behaviour and verification
6	UI/UX specification	Defines screens, interactions, and accessibility
7	Implementation backlog	Converts the design into small coding tasks
8	Release checklist	Ensures required tests, permissions, and documentation are complete

The test plan should begin alongside the PRD and become more detailed throughout development. UI/UX work can progress in parallel with the HLD once the key platform constraints are understood.

---

4. Proposed HLD outline

The HLD should contain these sections:

1. Purpose and scope.
2. Architecture principles.
3. System context diagram.
4. Component diagram.
5. Component responsibilities.
6. Usage detection and event flow.
7. Rolling-window budget and blocking flow.
8. Reminder scheduling and delivery flow.
9. Local persistence strategy.

10. Permission and lifecycle handling.
11. Offline operation.
12. Privacy and security.
13. Error handling and recovery.
14. Performance and battery considerations.
15. Calendar integration extension points.
16. Widget extension points.
17. Technical risks and unresolved decisions.

Avoid embedding implementation-level Kotlin code in the HLD. That belongs in the detailed design.

---

5. Proposed technical design outline

Usage engine

- Definition of a usage interval.
- Timestamp representation.
- Rolling-window algorithm.
- Five-minute grace-period logic.
- Cooldown calculation.
- Restart reset semantics.
- Temporary override semantics.
- Handling missing usage events.
- Unit tests.

Android integration

- Required permissions.
- Usage detection API.
- Blocking mechanism.
- Background execution.
- Notification and alarm strategy.
- Reboot recovery.
- Permission revocation.
- Supported Android versions.

Reminder engine

- Reminder data model.
- Scheduling and cancellation.
- Repeating-event rules.
- Snooze calculations.
- Acknowledgement state.
- Missed reminders.
- Silent and vibrate behaviour.
- Full-screen fallback.

Local persistence

- Reminder records.
- Selected applications.
- Usage history needed for the rolling window.
- Cooldown and override state.
- App preferences.
- Schema migrations.

Future integrations

- Calendar source identifiers.
- Event ownership and write-back.
- Synchronization conflict handling.
- Widget data queries and refresh.

---

6. Recommended implementation architecture

Use native Android development with Kotlin and Jetpack Compose.

Suggested boundaries:

- **UI layer:** Screens, navigation, blocker presentation, reminder presentation.
- **Domain layer:** Rolling-window budget, session continuity, cooldown, override, reminder policies.
- **Platform layer:** Android usage APIs, scheduling, notifications, permission handling, calendar provider.
- **Data layer:** Room database, DataStore, repositories.

Keep the core business rules independent of Android UI components so they can be tested with ordinary unit tests.

Do not add a backend, authentication, or a networking layer unless a later requirement actually needs it.

---

7. Vibe-coding workflow

Step 1 — Create the repository

Set up:

- Git repository.
- Android project.
- README with setup instructions.
- PRD and design documents.
- `DECISIONS.md`.
- `CHANGELOG.md` or a lightweight release log.

Make the initial project build before adding business logic.

Step 2 — Give the coding agent bounded tasks

Each task should specify:

1. The relevant PRD requirements.
2. The exact component being implemented.

3. Expected inputs and outputs.
4. Acceptance criteria.
5. Required unit or integration tests.
6. Constraints on unrelated changes.

Example task:

"Implement the rolling-window usage-budget calculator as a pure Kotlin component. It must calculate accumulated foreground usage in a trailing 60-minute window, preserve usage across session boundaries, and expose whether the 30-minute budget is exhausted. Add deterministic unit tests for the boundary conditions. Do not implement Android usage detection or UI in this task."

Step 3 — Review every generated change

After each task:

- Inspect the diff.
- Ask the agent to explain significant decisions.
- Run the relevant tests.
- Build the project.
- Commit a known-good state.

Do not let the coding agent silently rewrite unrelated components or introduce new libraries without justification.

Step 4 — Integrate one vertical slice at a time

A vertical slice is a small feature that works from user interaction through business logic to persisted state and back.

Examples:

- Select an app → save selection → display selection after restart.
- Create a reminder → persist it → schedule it → deliver it → acknowledge it.
- Reach usage limit → calculate block state → display blocker → expire cooldown → restore access.

Step 5 — Test on a physical device

Emulators are useful, but real devices are necessary for validating battery restrictions, background execution, notification behaviour, permissions, and manufacturer-specific differences.

Step 6 — Keep the documents current

Whenever an implementation decision changes:

- Update the technical design.
- Update `DECISIONS.md`.
- Update acceptance criteria if the product behaviour changes.
- Record any remaining limitations.

---

8. Testing strategy

Unit tests

- Rolling-window calculations.
- Session grace-period boundaries.
- Cooldown expiration.

- Temporary override duration.
- Reminder recurrence.
- Snooze calculations.
- Reminder state transitions.

Integration tests

- Usage events feeding the domain engine.
- Persistence across process death.
- Scheduling and cancellation.
- Reminder acknowledgement.
- Permission-state changes.
- Reboot recovery.

Device tests

- Screen off and screen on.
- Rapid app switching.
- Switching away for less than five minutes.
- Switching away for more than five minutes.
- Usage split across multiple sessions.
- Reopening through app links.
- Silent and vibrate modes.
- Battery saver.
- App force-stop.
- Device restart.
- Offline operation.

Regression tests

Every fixed defect should result in a reproducible test where practical. The most important protection is to ensure that future code changes do not break the rolling budget, cooldown, or reminder delivery.

---

9. Decision log

Record significant decisions with their rationale and consequences (see `DECISIONS.md` for full register).

Decision	Current position	Consequence
Platform	Android-first (API 29–35)	Supports current latest (Android 15) and 5 versions back (>90% device coverage)
Architecture	Local-first	No backend needed for core functionality; offline privacy guaranteed
Rolling usage budget	Configurable (default 30m in rolling 60m)	User flexibility while preserving strict pacing defaults
Daily total limit	Configurable per app, midnight reset	Cumulative daily cap prevents pacing exploits across the full day
Session grace period	Five minutes	Defines session continuity, not extra usage
Minimum cooldown	30 minutes	Access cannot return immediately after the threshold
Restart	Reset active tracking state	Accepted MVP bypass risk
Emergency access	Written reflection, user-set duration, full repeat	User inputs needed time; re-locks on expiry; requires full 5-question repeat
Reflection storage	Retained locally for user review	Saved in local SQLite database; never processed or transmitted
Reminder duration	Selective: P0 sticky; P1/P2 timed	P0 requires explicit acknowledgement; P1/P2 dismisses to notification
Reminder fallback	High-priority system notification	Guaranteed alert when full-screen overlay is constrained by OS
Browser monitoring	Excluded	Preserves privacy and reduces scope
Calendar	Google Calendar only; Phase 2	Deferred post-MVP to keep initial release tight
Widget	Later phase (Phase 4)	Reuses the established data layer

10. Definition of done for the MVP

The MVP is complete when:

- The P0 requirements in the PRD are implemented.
- The rolling-window engine passes deterministic tests.
- Usage tracking and blocking work on the defined physical-device test matrix.
- Emergency access is temporary and follows the documented rules.
- Local reminders can be created, rescheduled, acknowledged, and snoozed.
- Reminder delivery and fallback behaviour are verified.
- Silent and vibrate behaviour respects system settings.
- Core functionality works offline.
- Privacy and permission behaviour are documented.
- Known limitations are visible to the user where relevant.
- Critical defects are resolved.
- The repository builds reproducibly and has setup instructions.

11. Immediate action checklist

1. Review and approve the PRD.
2. Resolve the open decisions that affect the MVP.
3. Create the Git repository and native Android project.
4. Build the usage-detection feasibility prototype.
5. Build the full-screen-reminder feasibility prototype.
6. Document findings and platform constraints.
7. Write the HLD based on demonstrated capabilities.
8. Write the detailed technical design for the usage engine and reminder engine.
9. Create the first implementation backlog.
10. Build and test one vertical slice at a time.
11. Release a personal-use MVP.
12. Add calendar integration.
13. Add the daily-planner widget.
14. Prioritise future features based on observed usage and feedback.

> **FINAL PRINCIPLE**
>
> **The documents are not paperwork to finish before coding. They are living specifications that reduce uncertainty. Write enough to make the next decision correctly, validate the riskiest assumptions early, and only then invest in the next layer of detail.**